

INF01151 – Sistemas Operacionais II

Aula 23 - Deadlocks

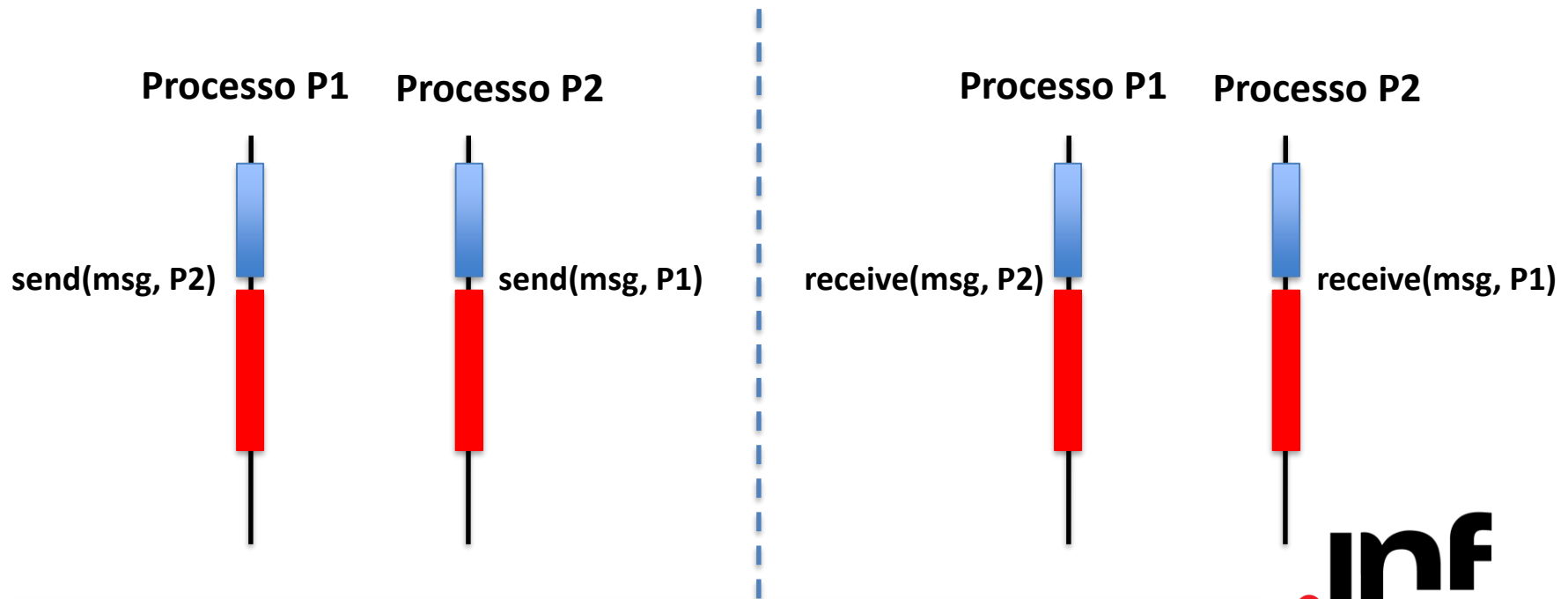
Prof. Alberto Egon Schaeffer Filho

Deadlock em sistemas distribuídos

- Deadlock é a espera por um evento que não ocorrerá
 - Situação em que processos/threads ficam bloqueados permanentemente esperando pela liberação de um recurso
- Deadlock distribuido
 - Ocorre quando processos dispersos por diferentes computadores de uma rede aguardam por eventos que não ocorrerão
 - Problema similar a sistemas centralizados
 - Dificuldade é a coleta e a manutenção de informações
- Certos autores dividem em:
 - *Deadlocks* de comunicação
 - *Deadlocks* na alocação de recursos (exemplo clássico!)
 - *Deadlocks* fantasmas

Deadlock de comunicação

- Devido à semântica bloqueante de send / receive
 - Uma espera circular por comunicação
 - P_1 bloqueia no envio de mensagem a P_2 , que bloqueia no envio de mensagem a P_3 , que bloqueia no envio de mensagem a P_1
 - P_1 espera uma resposta de P_2 , que espera uma resposta de P_3 , que espera uma resposta de P_1



Condições para ocorrência de deadlock

- Quatro situações devem ser satisfeitas simultaneamente:

1. Exclusão mútua

- Somente um processo pode usar o recurso em um determinado instante

2. Segura/espera

- Um processo, de posse de um recurso, espera para alocar outro recurso

3. Recurso não-preemptível

- Uma vez alocado um recurso, ele não pode ser retirado de um processo

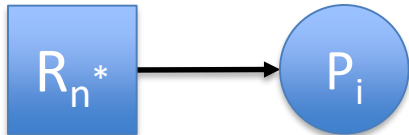
4. Espera circular

- Uma sequência de processos onde um processo mantém ao menos um recurso que é necessário ao próximo processo

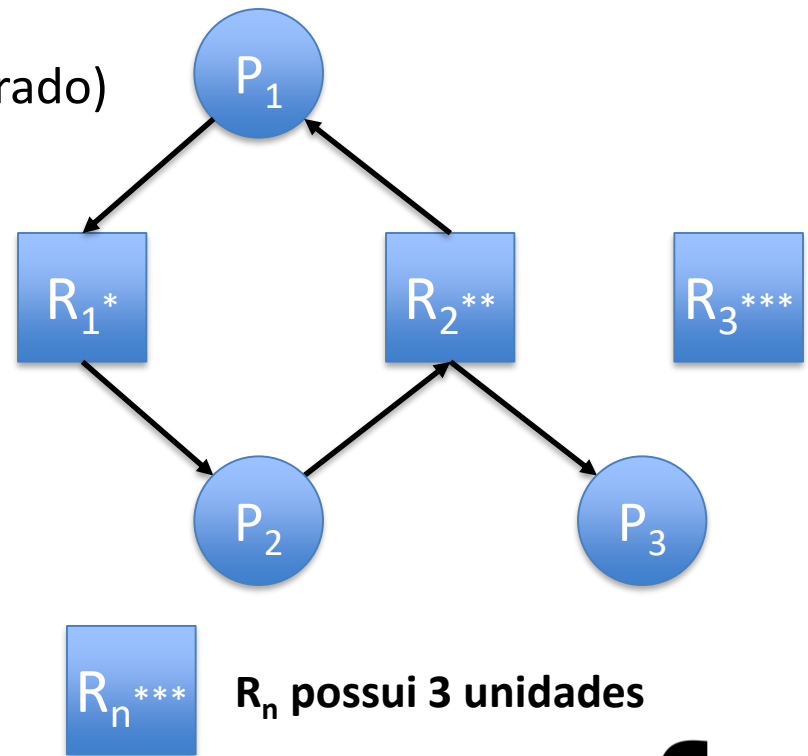
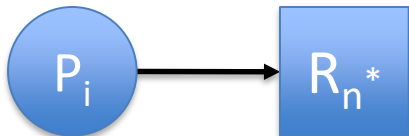
Se uma for quebrada, então *deadlock* não acontece no sistema!

Modelagem de deadlocks

- Grafo direcionado
 - Grafo de alocação e requisição de recursos
- Dois tipos de nós
 - Processos (círculo) e recursos (quadrado)
- Dois tipos de arestas:
 - Alocação (recurso $\rightarrow$ processo): processo detém um recurso

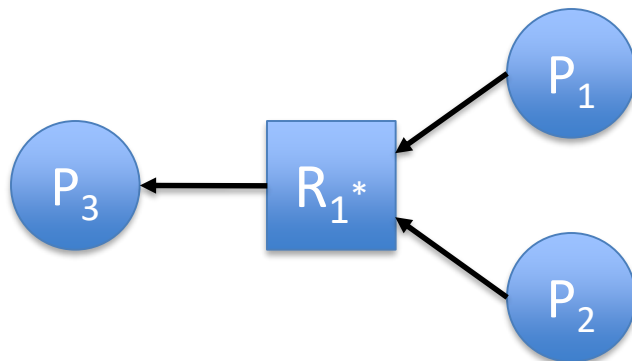


- Requisição (processo $\rightarrow$ recurso): processo deseja um recurso

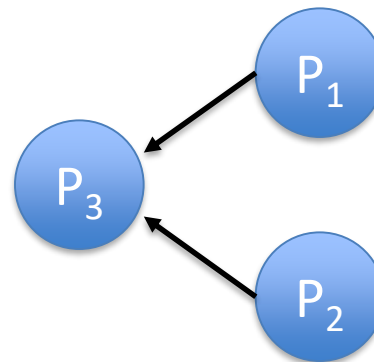


Grafos RRAG e WFG

- Resource Request and Allocation Graph (RRAG)
 - Dois tipos de nós: processos e recursos
- Wait-For-Graph (WFG)
 - Caso particular de um RRAG onde há apenas uma unidade de um determinado recurso
 - Simplificação, com apenas um tipo de nó: processos



RRAG



WFG

Tratamento de deadlock

- Ignorar
 - Algoritmo da “avestruz”
- Prevenir (*prevention*)
 - Condicionar o sistema a afastar qualquer possibilidade de deadlocks
 - “Negar” uma das quatro condições necessárias como no caso centralizado
- Evitar (*avoidance*)
 - Permitir que possibilidade exista, mas na iminência de acontecer, deadlock é cuidadosamente desviado
 - Alocação criteriosa de recursos (algoritmo do banqueiro como no caso centralizado)
- Detectar & recuperar
 - Deixar o deadlock ocorrer, e ser capaz de determinar sua existência
 - Depois, limpar os deadlocks do sistemas
 - Mecanismos de *checkpoint*, transação e suspensão/retomada



Prevenção de deadlock

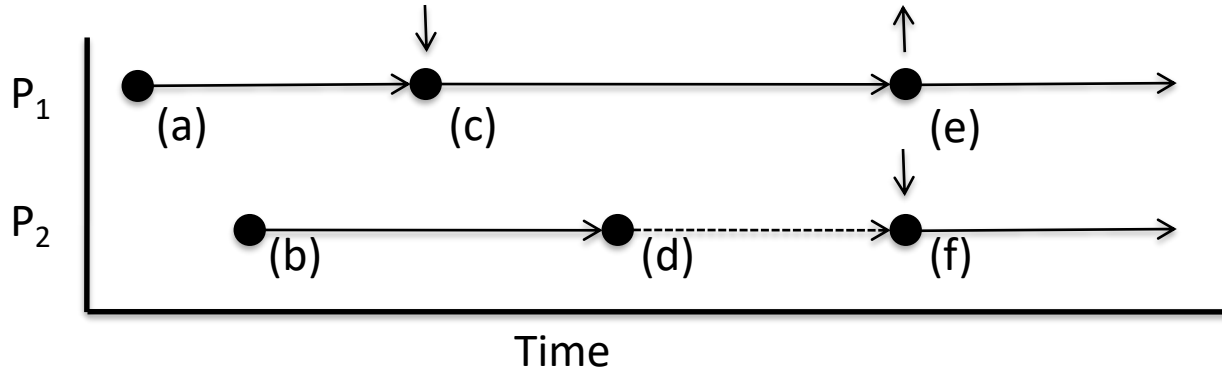
- Objetivo:
 - Eliminar uma das quatro condições necessárias
- Duas estratégias básicas para prevenir deadlocks [Rosenkrantz, 1978]
 - *Wound-wait* (ferir-esperar)
 - *Wait-die* (esperar-morrer)
- Ambos os algoritmos dependem da ordenação de processos com base no momento em que cada processo foi iniciado
 - Processos ordenados por idade
 - Processos na criação recebem timestamps
 - $\langle \text{tempo_local}, \text{id_nó} \rangle$

Estratégias *wound-wait* e *wait-die*

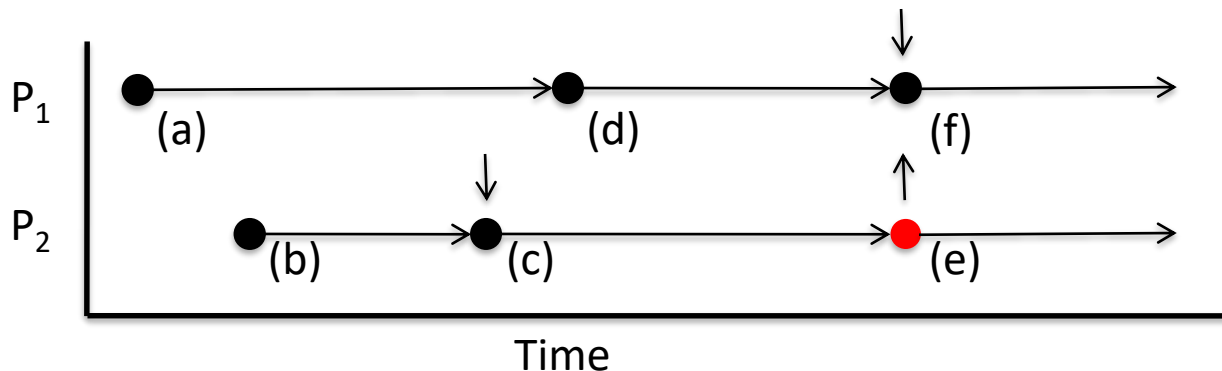
- *Wound-wait*

- Processo mais novo (maior *timestamp*) pode esperar pelo mais velho, mas o mais velho não espera pelo mais novo

- Nega condição de recurso não-preemptível



- (a) P_1 starts
- (b) P_2 starts
- (c) P_1 acquires R_1
- (d) P_2 requests R_1 , waits
- (e) P_1 releases R_1
- (f) P_2 acquires R_1



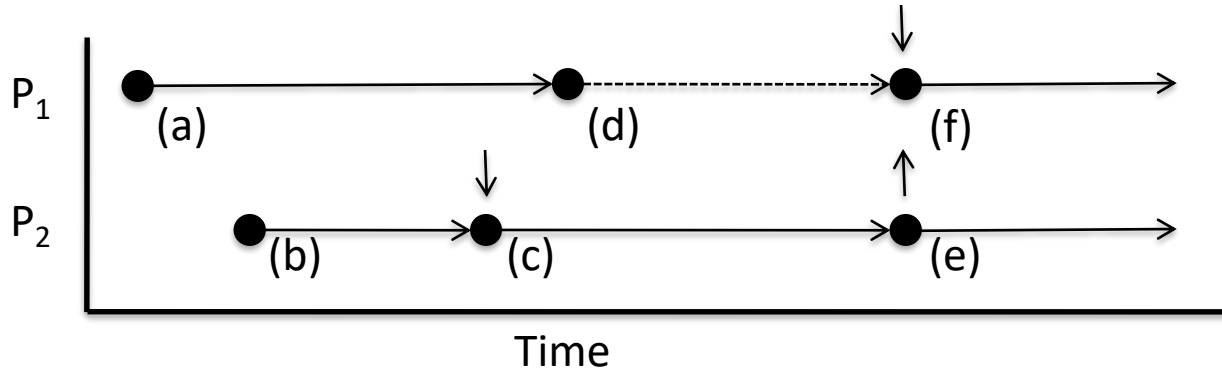
- (a) P_1 starts
- (b) P_2 starts
- (c) P_2 acquires R_1
- (d) P_1 requests R_1 , wounds P_2
- (e) P_2 is restarted, releases R_1
- (f) P_1 acquires R_1

Estratégias *wound-wait* e *wait-die*

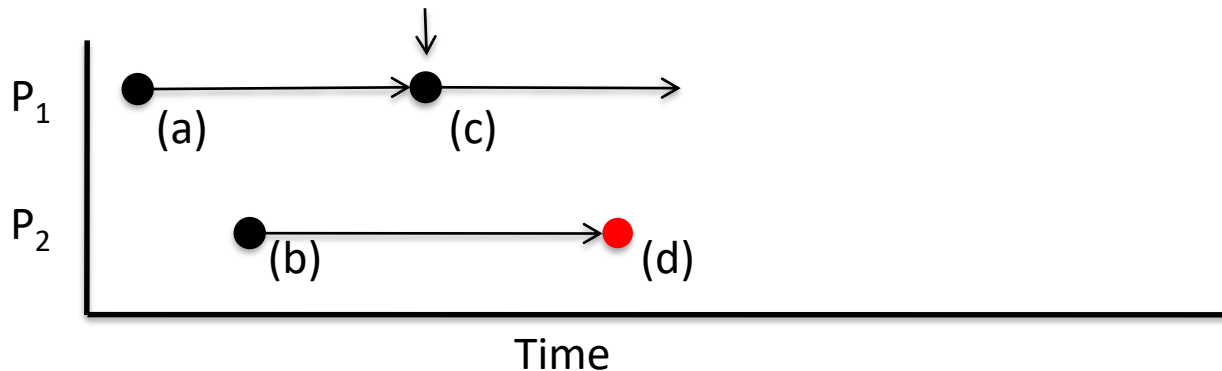
- *Wait-die*

- Processo mais velho (menor *timestamp*) pode esperar pelo mais novo, mas o mais novo não espera pelo mais velho

- Nega condição de segura/espera



- (a) P_1 starts
- (b) P_2 starts
- (c) P_2 acquires R_1
- (d) P_1 requests R_1 , waits
- (e) P_2 releases R_1
- (f) P_1 acquires R_1



- (a) P_1 starts
- (b) P_2 starts
- (c) P_1 acquires R_1
- (d) P_2 requests R_1 , dies

Questão do aborto de processo

- As estratégias abortam, se necessário, o processo mais novo
 - O que realizou menos tarefas (redução de prejuízo)
 - Wound-wait:
 - Processo novos são reiniciados se recurso for requisitado por processo mais velho
 - Processos novos devem esperar por processos mais antigos
 - Conforme envelhece, esperam com menos frequência
 - Wait-die:
 - Processos novos irão morrer ao invés de esperar
 - Conforme processos envelhecem, podem ser forçados a esperar com mais frequência
 - Cuidados: preemptar recursos alocados antes de abortar o processo

Situação	<i>Wait-die</i>	<i>Wound-wait</i>
P_{velho} precisa recurso do P_{novo}	P_{velho} espera	P_{novo} aborta
P_{novo} precisa recurso do P_{velho}	P_{novo} aborta	P_{novo} espera

Detecção de deadlocks

- Deixar o deadlock ocorrer, e ser capaz de determinar sua existência
 - Consiste em detectar ciclos em grafos de alocação e removê-los
 - Quando detectado, selecionar um processo para ser abortado para quebrar o ciclo
 - Quem escolher para abortar?
- Técnicas similares ao caso centralizado
 - Porém, no caso centralizado existe um único grafo de alocação com uma representação “global”
- Algoritmos para sistemas distribuídos:
 - Centralizado
 - Distribuído (Chandy-Misra-Hass)

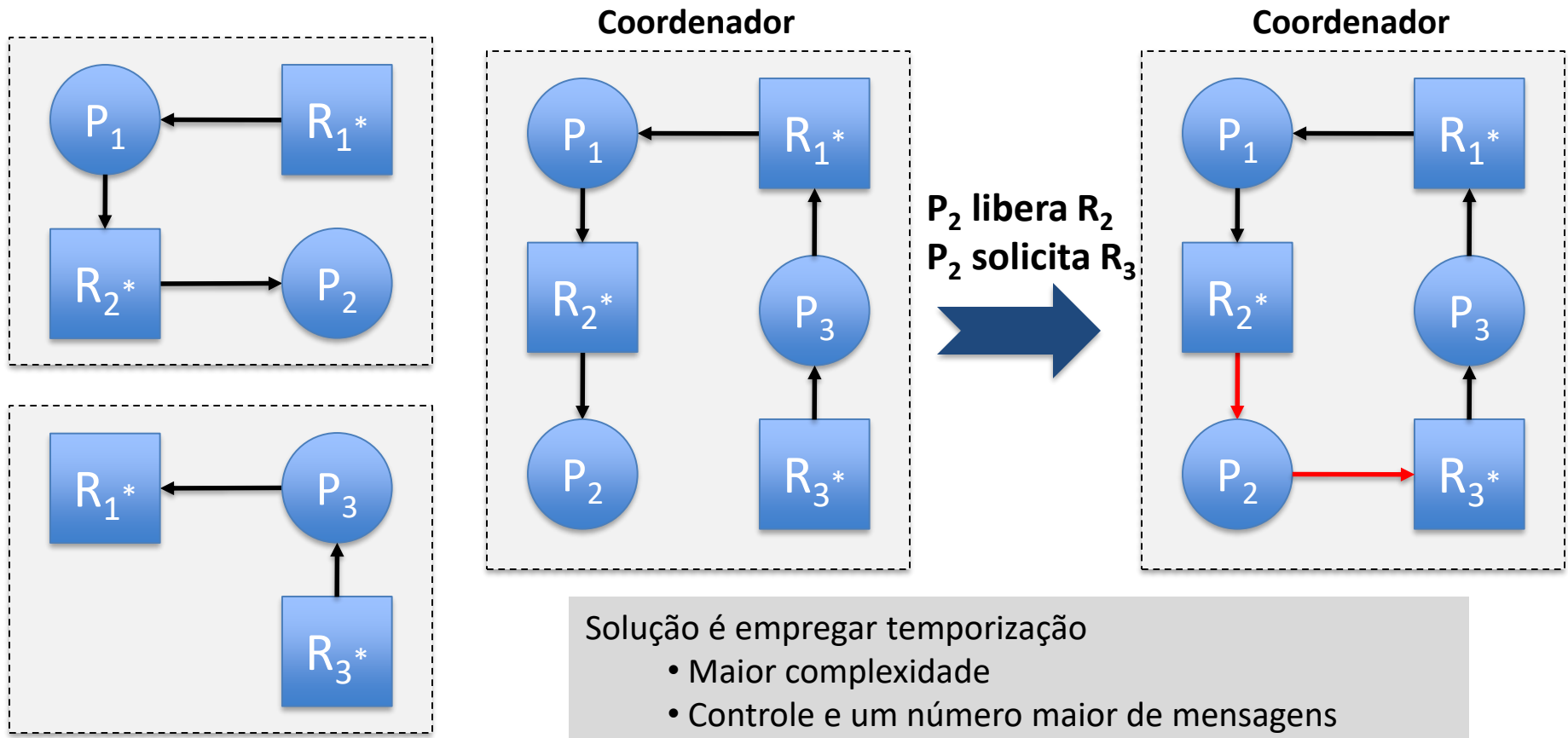
Algoritmo centralizado

- Cada computador mantém grafo local de alocação de recursos
- Existe um computador central (coordenador) que monitora o sistema inteiro
 - Outros processos notificam o coordenador quanto a alocações
 - Coordenador constrói um grafo de alocação “global”
 - Quando um ciclo for detectado
 - Coordenador implementa algum algoritmo de recuperação (e.g., matar um processo)
- Opções para construção do grafo global:
 - ① Mensagens de atualização para cada adição ou remoção de arestas
 - ② Periodicamente, com envio de lista de arestas adicionadas/removidas
 - ③ Sob demanda do coordenador

Problemas usuais: baixa escalabilidade, ponto unico de falhas, trade-off sobre a frequência de updates, ...

Problema: deadlock fantasma

- Devido a atrasos associados à computação distribuída, é possível que um algoritmo detecte um deadlock que não exista
 - Deadlock fantasma: é “detectado” mas não é na verdade um deadlock



Algoritmo Chandy-Misra-Haas

- Algoritmo distribuído (edge chasing, path pushing)
 - Grafo global de alocação não é contruído
 - Mensagens de probe enviadas através do grafo por todo o sistema distr.
- Princípio de funcionamento:
 - Mensagens especiais (*probe*) são enviadas pelo grafo de dependência
 - Ao receber um *probe*, o processo envia a seu sucessor
 - Se um *probe* retorna ao seu originador: → *deadlock*

Probe é uma tupla (*i*, *j*, *k*)

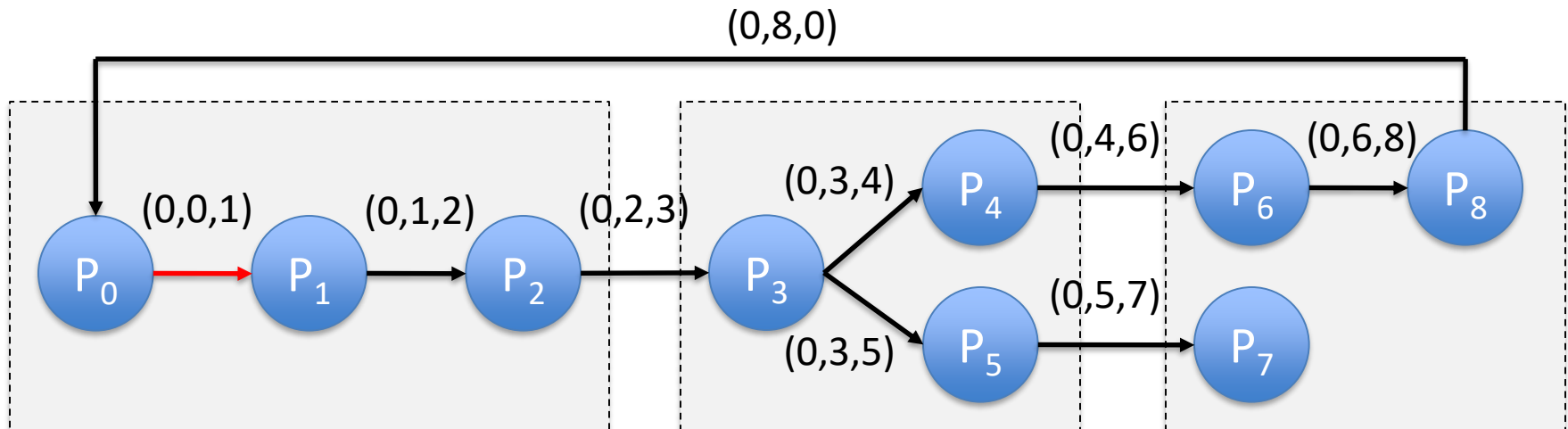
i: iniciador do processo de detecção de *deadlock*

j: emissor do *probe*

k: destinatário do *probe*

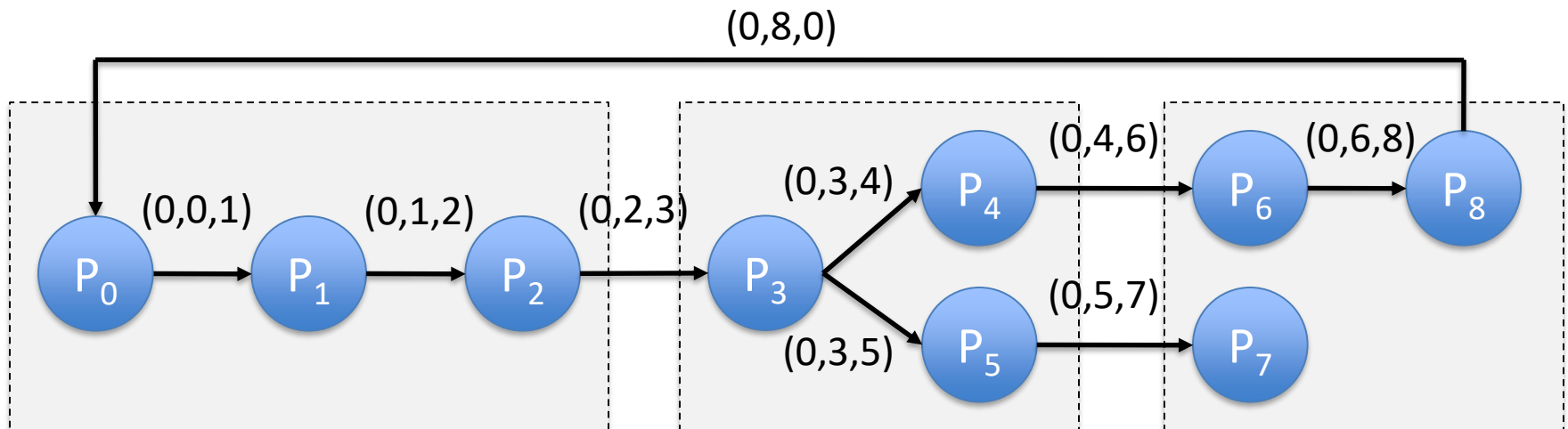
Algoritmo Chandy-Misra-Haas

- Funcionamento do algoritmo
 - Sempre que um processo P_i será bloqueado devido a outro processo (P_j), ele envia uma mensagem com o *probe*(i, i, j)
 - Se P_j recebe *probe*(i,i,j) e P_j está bloqueado a espera de um processo P_k , então ele envia uma mensagem *probe*(i,j,k) para P_k
 - Se P_j não está bloqueado, nenhuma mensagem é enviada
 - Se o *probe* retornar à origem (P_i) é porque existe um *deadlock*



Algoritmo Chandy-Misra-Haas

- Como quebrar o *deadlock*?
 - Processo que iniciou a seqüência de mensagens se suicida
 - Desvantagem (risco): suicídio em demasia devido ao tempo da mensagem circular
 - E.g.: P_6 e P_0 iniciam o *probe* ao mesmo tempo
 - Incluir na mensagem a identidade de quem a envia
 - O emissor original recebe uma lista dos processos envolvidos no ciclo
 - Eliminar o de mais alto (baixo) identificador/prioridade/timestamp



Recuperação de deadlock

- Geralmente realizada pela retirada forçada de um processo do sistema e retomada de seus recursos
 - A idéia é “voltar” o sistema para uma situação onde não ocorra o *deadlock*
 - Envolve suspensão, morte ou reinicialização de processo(s)
 - Pode implicar em perder o trabalho que o processo retirado realizou
 - Como determinar qual processo retirar?
- Técnicas básicas (mesma de centralizados)
 - Suspensão/retomada (preempção temporária dos seus recursos)
 - Quando for seguro, retomar o processo retido sem perda de trabalho
 - Checkpoint
 - Limita a perda de trabalho até o momento do último checkpoint
 - Transações
 - Mudanças permanentes se transação for concluída com sucesso

Leituras adicionais

- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 7
 - Capítulo 18 (seção 18.5)
- Deitel, H.; Deitel, P.; Choffnes, D. – *Sistemas Operacionais*, 3ª edição, Pearson Prentice Hall, 2005.
 - Capítulo 7
 - Capítulo 17 (seção 17.6)
- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G. – *“Distributed Systems: Concepts and Design”* (5th edition), Addison Wesley, 2012.
 - Capítulo 16 (seção 16.4)
 - Capítulo 17 (seção 17.5)